Reviewer Pack — Minimal Rank of Algebraic K-Theory Groups over Tseitin Resol…

Entry 5c394995c20e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 22:42:54 UTC

Minimal Rank of Algebraic K-Theory Groups over Tseitin Resolution Trees

Entry ID: `5c394995c20e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 22:42:54 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

> ['For every Tseitin resolution tree T for a CNF formula F with n variables, the minimal rank of the associated algebraic K-theory group is Θ(n log n).', 'Equivalently, there exists an absolute constant c > 0 such that for all Tseitin resolution trees T, |K_1(T)| ≥ c n log n.', 'Where |K_1(T)| denotes the rank of the first algebraic K-group of T.']

Rationale (proposer's reasoning):

> ['Algebraic K-theory has been used in other areas of complexity theory to study properties of quantum circuits and topological quantum computing. This conjecture suggests that it could provide a new perspective on resolution complexity, which is a fundamental problem in complexity theory.', 'If true, this would imply a strong connection between the structure of Tseitin resolution trees and the underlying algebraic K-theory groups, potentially leading to new algorithms or insights into the nature of computational problems.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7f0cf59a65e307b3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given CNF formula F with n variables, the associated algebraic K-theory group's minimal rank |K_1(T)| will be considered supported if it meets both conditions: (1) The mean rank across all 30 trees is greater than or equal to c * n * log(n) for some absolute constant c > 0, and (2) The standard deviation of the ranks does not exceed a predefined threshold. Falsified if any single tree's |K_1(T)| is less than c * n * log(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `algebraic K-theory AND Tseitin resolution trees` - `minimal rank K_1(T) AND CNF formula` - `Tseitin resolution trees AND complexity theory`

Top relevant hits considered: - [http://arxiv.org/abs/1208.1599v1] Algebraic K-theory of endomorphism rings - [http://arxiv.org/abs/2104.12736v2] Deformation theory of perfect complexes and traces - [http://arxiv.org/abs/1006.1480v3] Reduced Steenrod operations and resolution of singularities - [http://arxiv.org/abs/2411.07730v2] Study of the light scalar $a_0(980)$ through the decay $D^0 \to a_0(980)^- e^+ \nu_e$ with $a_{0}(980)^- →ηπ^- - [http://arxiv.org/abs/2303.12927v2] Study of the $f_0(980)$ and $f_0(500)$ Scalar Mesons through the Decay $D_s^+ \to \pi^+\pi^- e^+ \nu_e$ - [http://arxiv.org/abs/hep-ph/0610217v3] Clues for the existence of two $K_1(1270)$ resonances - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly's Eye Exp - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][i] == 0:
            raise ValueError("Matrix is singular")
        for j in range(cols):
            matrix[i][j] /= matrix[i][i]
        for k in range(rows):
            if k != i and matrix[k][i] != 0:
                factor = matrix[k][i]
                for j in range(cols):
                    matrix[k][j] -= factor * matrix[i][j]
    return matrix
```

```python
def rank(matrix):
    row_echelon_form = gaussian_elimination([row[:] for row in matrix])
    non_zero_rows = [row for row in row_echelon_form if any(row)]
    return len(non_zero_rows)

def generate_cnf(n):
    literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]
    clauses = []
    for _ in range(n):
        clause = random.sample(literals, random.randint(2, n))
        clauses.append(clause)
    return clauses

def tseitin_resolution_tree(cnf):
    literals = set()
    for clause in cnf:
        literals.update(clause)
    nodes = {var: [] for var in literals}
    for clause in cnf:
        new_var = len(nodes) + 1
        nodes[new_var] = []
        for literal in clause:
            if literal > 0:
                nodes[literal].append(new_var)
                nodes[-literal].append(-new_var)
            else:
                nodes[-literal].append(new_var)
                nodes[literal].append(-new_var)
    return nodes

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    tree = tseitin_resolution_tree(cnf)
    k_group_rank = rank([[0] * len(tree) for _ in range(len(tree))])
    metric_value = k_group_rank
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    if n > 1:
        c = Fraction(1, 2)
        lower_bound = c * n * math.log(n)
        if metric_value >= lower_bound and abs(metric_value - lower_bound) / lower_bound <= 0.1:
            conjecture_holds = True

    return {
        "metric_name": "K-group rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
```

```python
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) + [101, 103, 107, 109, 113]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...{result}...}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_faili
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support support_fraction={support_fractio
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71e04ede.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71e04ede.py", line 69, in run_trial
    k_group_rank = rank([[0] * len(tree) for _ in range(len(tree))])
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71e04ede.py", line 35, in rank
    row_echelon_form = gaussian_elimination([row[:] for row in matrix])
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71e04ede.py", line 24, in gaussian_elim
    raise ValueError("Matrix is singular")
ValueError: Matrix is singular
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not provide any evidence to support or falsify the conjecture. | next: Re-run the test with appropriate error handling and analysis of the results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12205
2	propose	ollama_remote	glm4:latest	0	0	10566
3	preregistration	ollama_remote	glm4:latest	0	0	6432
4	novelty	ollama_remote	glm4:latest	0	0	4554
5	novelty	ollama_remote	glm4:latest	0	0	6516
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43259
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7024
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11075
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11520
10	judge	ollama_remote	glm4:latest	0	0	11634

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 124784 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5c394995c20e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5c394995c20e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5c394995c20e.tar.gz` (if generated)